

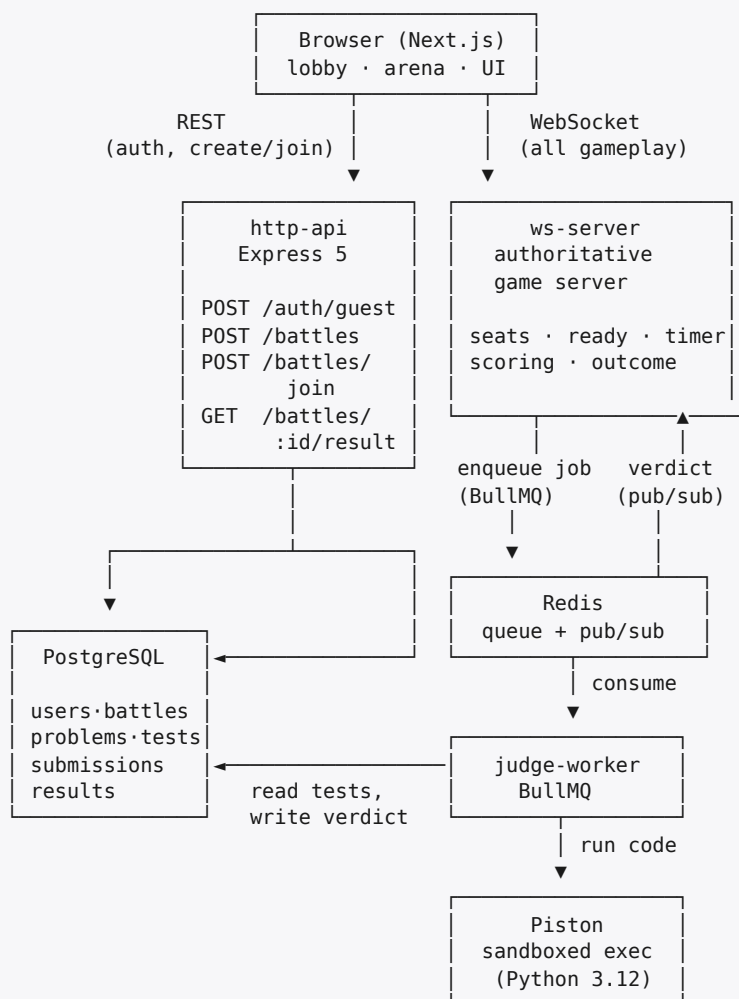
Technical Approach & Architecture

CombatX — real-time competitive coding battles

Document 1 of 3 · [Why/What/How](#) · [User Guide](#)

1. System overview

CombatX is a pnpm/Turborepo monorepo containing four runtime services and five shared packages. The browser holds two connections: REST to `http-api` for transactional work, and a long-lived WebSocket to `ws-server` for the battle.



Service responsibilities

Service	Owns	Explicitly does <i>not</i>
<code>web</code>	Rendering, input, optimistic UI	Any authoritative state
<code>http-api</code>	Guest auth, battle creation/join, result reads	Live gameplay
<code>ws-server</code>	All game state: seats, readiness, timer, scoring, outcome	Executing code
<code>judge-worker</code>	Running submissions, producing verdicts	Deciding who wins

The separation between the last two is the core design decision: **the referee never executes code, and the executor never decides outcomes.**

2. The authority model

Every piece of contestable state lives on `ws-server`. Clients send *intent*; the server decides what actually happened.

Client sends	Server decides
<code>team:select</code>	Whether that seat is free and the battle is open
<code>player:ready</code>	Whether the lobby is now startable
<code>battle:start</code>	Whether the requester is the host and rules pass
<code>code:submit</code>	The submission's server-side timestamp

A client cannot mark itself ready, award itself a point, set its own submission time, or end a battle. There is no client-supplied timestamp anywhere in the scoring path — `submittedAt` is stamped on receipt by the server and is, per `packages/game/src/outcome.ts`, *"the sole tie-break authority."*

Why this matters for scoring

The outcome rule is: **highest passed-count wins; ties break by who reached that count first.** If clients supplied their own timestamps, every tie would be winnable by lying. Server-stamped receipt time closes that.

Deterministic problem assignment

Problems are not chosen randomly per battle. `battleRoom.ts` selects from the candidates for the room's difficulty using **`battleId` as the seed**, so the assignment is reproducible: every participant provably receives the same problem, and a battle can be replayed or audited without storing the choice separately.

3. Wire protocol

All frames are Zod-validated in both directions (`packages/protocol`). An invalid frame is rejected, not coerced.

Client → server: `hello` · `team:select` · `player:ready` · `battle:start` · `code:submit` · `battle:leave` · `ping`

Server → client: `snapshot` · `lobby:update` · `presence:update` · `battle:countdown` · `battle:start` · `timer:tick` · `submission:queued` · `submission:result` · `opponent:progress` · `battle:finished` · `ack` · `ack-error` · `error` · `pong`

The opponent-safety boundary

`opponent:progress` is documented in `packages/protocol/src/messages.ts` as *"The ONLY progress signal opponents receive (no source)." It carries a passed-count and nothing else — no code, no test names, no failure output.*

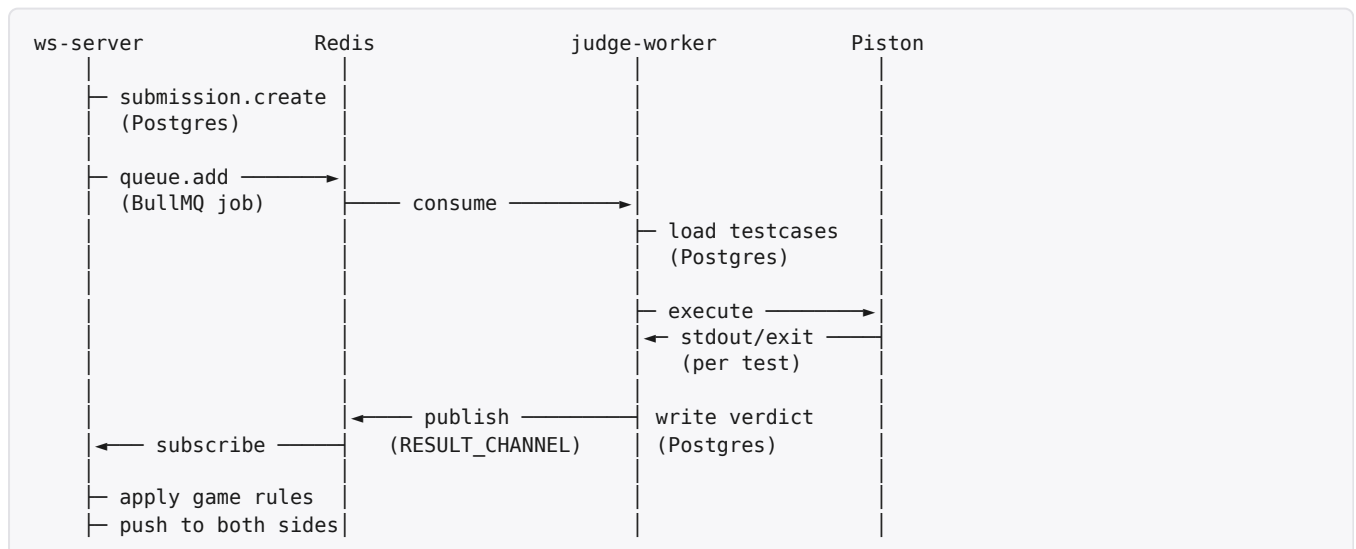
This is enforced by the schema, not by convention. There is no field on the opponent-facing message that *could* carry source, so a future change that tries to leak it fails type-checking in both the client and the server.

Reconnection

`snapshot` carries full current state, letting a client that drops mid-battle rebuild without special-case recovery logic. The server is the single source of truth, so replaying state is just sending it again.

4. The judging pipeline

Executing untrusted code is both **dangerous** and **slow**. Those are two different problems with two different solutions.



Danger → **isolation**. Code runs in a Piston container, never in the app processes. `judge-worker` is the only service that talks to it.

Slowness → **decoupling**. Execution takes hundreds of milliseconds per test. A game server that blocks on that stops sending timer ticks to *everyone in the room*. So submissions become BullMQ jobs; the socket path stays hot regardless of judge load, and workers scale independently (`JUDGE_CONCURRENCY`).

Defensive limits

`apps/judge-worker/src/judge/piston.ts` applies:

- `run_timeout` / `compile_timeout` — passed to Piston per execution
- an `AbortController` on the HTTP call (`httpTimeoutMs`) so a hung sandbox can't wedge a worker
- **output truncation** — commented in-source as ensuring *"one runaway print can't blow up memory"*

Jobs use `jobId: submissionId` , making enqueue idempotent: a retry or duplicate frame cannot double-judge one submission.

5. Data model

Eight Prisma models (`packages/db/prisma/schema.prisma`):

```
graph LR
    User <-- TeamMember <-- Team <-- Battle
    Problem <-- TestCase
    Battle --> Submission
    Submission --> Result
```

Model	Role
<code>User</code>	Guest identity. No password field — guest-only
<code>Problem</code>	Statement and metadata
<code>TestCase</code>	Hidden inputs/outputs. Read only by <code>judge-worker</code>
<code>Battle</code>	Room, mode, lifecycle state, timing
<code>Team</code>	Side A or B within a battle
<code>TeamMember</code>	Seat: which user on which team
<code>Submission</code>	One attempt, with server-stamped <code>submittedAt</code>
<code>Result</code>	Final standings and deciding submission

Battle lifecycle: `LOBBY` → `IN_PROGRESS` → `FINISHED` .

Guest-only play is **structural, not policy** — there is no credential column to misuse, so there is no password to leak.

6. Type and validation strategy

Two shared packages are compiled into both the frontend and the backend:

- `packages/protocol` — Zod schemas for every REST body and socket frame
- `packages/game` — pure functions for lobby readiness, scoring, and outcome

This is what prevents client/server drift. A protocol change that breaks a client surfaces as a **type error at build time**, not as a mystery bug during someone's tournament.

Keeping the rules pure also makes the part that decides winners the part that's easiest to verify:

`packages/game` is unit-tested in isolation, no database, no sockets (9 tests in `outcome.test.ts`).

Validation is applied at *every* boundary — REST bodies, socket frames in both directions, and jobs pulled off the queue (`JudgeJob.parse`). Nothing is trusted because it arrived over a channel that was trusted a moment ago.

7. Deployment

Each app ships two Dockerfiles:

Variant	Build	Purpose
<code>Dockerfile.dev</code>	3 stages, source bind-mounted	Hot reload
<code>Dockerfile.prod</code>	5–6 stages, <code>turbo prune</code> , non-root user	Slim, reproducible runtime

`turbo prune` isolates each app's dependency subtree, so an unrelated app's change doesn't invalidate the install cache. The result is measurable — `web` is **331 MB** in prod versus **1.19 GB** in dev.

One-command startup

Two init containers make `docker compose up` sufficient:

- `db-init` — applies the schema, seeds problems
- `piston-init` — installs the Python 3.12 runtime into the sandbox

Both are **idempotent** (the provisioner checks the installed-runtime list before POSTing) and both are gated behind healthchecks, so no app service can start against an unmigrated database or an empty sandbox.

8. Known constraints

- **1v1 is what ships.** 2v2–4v4 exist in schema, protocol, and rules, but are gated in the UI pending a larger-roster lobby flow.

- **Python only.** Adding a language means adding it to `PISTON_RUNTIME` in `packages/protocol/src/enums.ts` *and* the provisioner list — they must match.
- **Single ws-server instance.** Battle state is in-process. Horizontal scaling would need room affinity or state extraction to Redis.
- **No rate limiting on submissions** beyond queue concurrency.

Why, What & How

CombatX — real-time competitive coding battles

Document 2 of 3 · [Technical Approach](#) · [User Guide](#)

WHY

The problem

Competitive programming today has two shapes, and neither one is *a match you can start right now with someone you know*.

Practice sites are single-player

LeetCode, HackerRank, and their peers are asynchronous by design. You pick a problem, you solve it, you get a green check. There is no opponent, no clock that belongs to anyone but you, and no reason to be **first**.

That trains one skill — correctness. But a contest tests a different one: correctness *under pressure*, with someone else's progress visible in the corner of your eye. You cannot practice that alone, and these platforms never ask you to.

Contest platforms are scheduled and serious

Codeforces and CodeChef do have real competition. But rounds start at fixed times, run for two hours, and rank you against thousands of strangers on a rating ladder that follows you around.

That's a genuine sport and it serves people well. It just isn't what a university coding club wants on a Thursday evening, or what two friends want when one says *"bet you can't solve this faster than me."*

The gap

Nothing fills the space between them: **short, spontaneous, head-to-head**. Same problem, both players, one clock, someone wins, done in ten minutes.

Groups improvise it today — a shared screen, a phone timer, and the honor system. It works badly. Someone reads the problem earlier than someone else. Nobody can verify the other person's tests actually passed. There's no record of who won.

Who this is for

Coding communities and events. Bootcamps, university clubs, Discord servers, internal team competitions — groups that already gather to code and want to run a bracket, a warm-up round, or a quick grudge match.

Two properties of that audience drive the entire design:

They gather in bulk, briefly. A club cannot ask forty people to create accounts before a session starts. So play is **guest-only** — type a name, share a room code, you're in. There is no registration, no email, no password. (In fact, there is no password *column* — see [Technical Approach §5](#).)

Their organizer is not a DevOps engineer. Whoever runs the event should not spend an afternoon wiring up a database, a queue, and a code sandbox. So the entire stack is **one command**, with schema, seed data, and language runtime all provisioned automatically on first boot.

What this is *not*

No ratings. No ladders. No matchmaking against strangers. No streak mechanics pulling you back tomorrow.

CombatX assumes **you already know who you're playing**. You're in a room together, physically or on a call. It's a tool for a group that has gathered — not a platform trying to keep you online.

WHAT

In one sentence

Two teams race to solve the **same** problem; pass every hidden test first for an instant win, or hold the highest passed-count when the clock expires.

The three phases

1. Lobby

Players join with a room code and take a seat on Team A or Team B, then ready up. The host can start once both sides are seated, equal in size, and ready.

2. Arena

The server reveals the same problem to both sides simultaneously and starts the clock. Players write code and submit. Each submission runs against the hidden tests in a sandbox, and the verdict streams back.

What you see: your own verdict, in full — which tests passed, what failed.

What your opponent sees: a number. How many tests you've cleared. Nothing else — not your code, not which tests, not your errors.

3. Results

Two ways a battle ends:

Reason	Condition
ALL_PASSED	A side passes every test — immediate win
TIMEOUT	Clock expires — highest passed-count wins

Ties on passed-count break by **who got there first**, using the server's receipt timestamp.

Current scope

1v1 works end to end — guest auth, lobby, live scoring, sandboxed judging, both win conditions, persisted results.

2v2 through 4v4 are modelled across the schema, protocol, and game rules, but gated in the UI pending a lobby flow for larger rosters.

Python 3.12 is the shipped language. The runtime list is protocol-driven, so adding more is a config change plus a provisioner entry.

HOW

Three constraints, three decisions

Every significant architecture choice follows from one of three properties of the problem. None of them are technology preferences.

Constraint 1 — A match needs a referee

The moment there's an opponent, every piece of state becomes contestable. Who's ready. Who submitted first. Whose clock it is. Who won.

If any of that lives on the client, it can be forged. A contest you can cheat isn't a contest.

→ **ws-server is authoritative.** Clients send *intent*, never facts. A client cannot mark itself ready, award itself a point, set its own submission time, or end the battle.

→ **The rules are pure functions.** `packages/game` decides readiness, scoring, and outcome with no database and no sockets — so the component that determines winners is the one that's easiest to test in isolation. It has unit tests covering both win conditions and the tie-break.

Constraint 2 — A contest requires hidden tests

If you can read the test cases, you can special-case them, and the score means nothing.

→ **Tests never leave the backend.** They live in Postgres and are read only by `judge-worker`. They are never sent to a client, not even the submitter's.

→ **Opponents get a count, not a signal.** The `opponent:progress` frame carries a passed-count and nothing else — enough to feel the race, not enough to reverse-engineer the problem. The schema has no field that *could* carry source code, so this is enforced at compile time rather than by discipline.

Constraint 3 — Untrusted code is dangerous *and* slow

These are two problems, and they need two different answers.

Dangerous → **isolate it.** Every submission is a stranger's code running on your machine. It executes in a **Piston** sandbox in a separate container, reachable only by the worker — with execution timeouts, an HTTP abort guard, and output truncation so one runaway `print` can't exhaust memory.

Slow → **get it off the socket.** Execution costs hundreds of milliseconds per test. A game server that blocks on judging stops sending timer ticks *to everyone in the room*. So submissions become **BullMQ jobs on Redis**; `judge-worker` consumes them and publishes verdicts back over pub/sub.

The socket path stays responsive no matter how many people hit Run at once, and judging capacity scales independently of the game server.

The invisible constraint — the two sides must agree

A subtler failure mode: the client and the server disagreeing about the rules *mid-tournament*.

→ **Shared, compiled contracts.** `packages/protocol` (Zod schemas for every frame) and `packages/game` (the rules) are imported by both the frontend and the backend. A protocol change that breaks a client is a **type error at build time**, not a mystery bug during someone's event.

Validation runs at every boundary — REST bodies, socket frames in both directions, and jobs pulled off the queue. Nothing is trusted merely because it arrived over a channel that was trusted a moment ago.

Why one command

The audience constraint from [WHY](#) shows up as an engineering requirement: an organizer must be able to host this without a DevOps afternoon.

That's why `docker compose up --build` is genuinely sufficient. Two init containers handle what would otherwise be manual setup — `db-init` applies the schema and seeds problems, `piston-init` installs the language runtime. Both are idempotent and gated behind healthchecks, so the first request can never hit an unigrated database or an empty sandbox.

It is the difference between a project someone can run and a project someone will actually use at an event.

Summary

The problem	The response
No quick head-to-head format exists	Room-code battles, ten minutes, someone wins
Groups can't onboard 40 people	Guest-only — no accounts, no passwords
Every match state is contestable	Authoritative server; clients send intent only
Readable tests make scores worthless	Hidden tests, backend-only; opponents see a count
Untrusted code is dangerous	Piston sandbox, separate container, hard limits
Untrusted code is slow	Queue-backed judging; socket path never blocks
Client/server can drift	Shared Zod protocol + rules, compiled into both
Organizers aren't DevOps engineers	One command, self-provisioning

User Guide

CombatX — real-time competitive coding battles

Document 3 of 3 · [Technical Approach](#) · [Why/What/How](#)

Part 1 — Running CombatX

Start it

You need **Docker** and nothing else.

```
git clone <repo-url> combatX
cd combatX
docker compose up --build
```

Open <http://localhost:3001>.

First boot takes a few minutes: it builds the images, applies the database schema, seeds the problems, and installs the Python 3.12 runtime into the code sandbox. All of that is automatic. Later starts take seconds.

You'll know it's ready when the logs settle and the page loads.

Stop it

```
docker compose down          # keep the database
docker compose down -v       # wipe the database too
```

If a port is already in use

The usual culprit is **Redis on 6379**, if you already run one locally.

```
cp .env.docker.example .env
echo "REDIS_PORT=6380" >> .env
docker compose up
```

Compose reads `.env` automatically. `POSTGRES_PORT` and `PISTON_PORT` work the same way. Everything has a working default, so only add the lines you need.

For running without Docker, see [SETUP.md](#).

Part 2 — Hosting a battle

Create a room

1. Open <http://localhost:3001>.
2. Enter a display name. That's the whole sign-up — no email, no password.
3. Create a battle. You'll get a **room code** (six characters, e.g. `UVPT2K`).
4. Share that code with your opponent.

Room codes deliberately avoid `0`, `o`, `1`, and `I`, so they're unambiguous when you read one out loud across a room.

Join a room

1. Open the site, enter a display name.
2. Choose to join, and enter the room code.

Anyone with the code can join. There's no invite list — this is designed for people who are already together in a room or on a call.

Start the match

In the lobby, both players:

1. **Pick a side** — Team A or Team B.
2. **Ready up**.

The host can start once both sides are seated, evenly sized, and ready. The server enforces this — you can't start a lopsided or half-ready battle.

A countdown runs, then both sides receive the **same problem** at the same moment.

Part 3 — Playing

The arena

Once the battle starts you'll see the problem statement, an editor, and a timer. Your opponent has exactly the same problem.

Write your solution and submit. Each submission is queued, run against the hidden tests in a sandbox, and the verdict comes back to you.

What you see, and what they see

You see	Your opponent sees
Your full verdict — passed/failed counts	Your passed-count only
Your own code	<i>Nothing</i> of your code

That asymmetry is deliberate. You get enough to feel the race — knowing they're at 4/5 while you're at 2/5 is the whole point — without anything you could use to copy their approach.

The hidden tests are never shown to anyone, including you. You see how many passed, not what they were.

Submitting

You can submit as many times as you like before the clock runs out. Only your **best** submission counts.

How you win

Outcome	What happened
ALL_PASSED	You passed every test — instant win , battle ends
TIMEOUT	Clock expired — highest passed-count wins

If both sides finish the clock on the same passed-count, the tie breaks in favour of whoever **reached that count first**, measured by when the server received the submission.

Practically: if you're stuck at 4/5 with a minute left, keep submitting — an earlier 4/5 beats a later one.

Results

When the battle ends both sides see the final standings: the winner, the reason, and each side's best passed-count. Results are saved, so the record survives a page refresh.

Part 4 — Reference

Ports

Service	URL
Web	http://localhost:3001
HTTP API	http://localhost:4001
WebSocket	<code>ws://localhost:4002/ws</code>
Piston	http://localhost:2000
Postgres	<code>localhost:5432</code> (loopback)
Redis	<code>localhost:6379</code> (loopback)

Current limits

- **1v1 only.** Team modes 2v2–4v4 exist in the data model and rules but are gated in the UI.
- **Python 3.12 only.**
- **Three seeded problems** ship by default: *Sum of Two Numbers*, *Reverse a String*, *Count Vowels*. Each battle is assigned one deterministically from its battle ID, so every player in a given room provably gets the same problem — and a room can be replayed identically.

Adding your own problems

Problems and their hidden tests live in `packages/db/prisma/seed.ts`. Add entries there, then re-run the seed:

```
pnpm --filter @repo/db db:seed
```

To browse or edit the data directly:

```
pnpm --filter @repo/db db:studio
```

Part 5 — Troubleshooting

The page won't load

Give it a minute on first boot — image builds plus provisioning take a while. Then check everything is up:

```
docker compose ps
```

All services should be `running`; `db-init` and `piston-init` should show `exited (0)` — that's success, they're one-shot setup containers.

Submissions hang or every test fails

The sandbox has no language runtime. Check:

```
curl http://localhost:2000/api/v2/runtimes
```

You should see Python 3.12. If the list is empty, `piston-init` didn't finish — check its logs:

```
docker compose logs piston-init
```

"Port is already allocated"

Something else is using one of the ports. See [If a port is already in use](#).

The battle won't start

The server refuses to start an invalid lobby. Confirm both sides are seated, the teams are the same size, and **everyone** has readied up.

I want to start completely fresh

```
docker compose down -v
docker compose up --build
```

`-v` drops the database volume, so the schema and problems are re-seeded from scratch.

Checking service health

```
curl http://localhost:4001/health # -> {"ok":true}
curl http://localhost:4002/health # -> {"ok":true}
```

Reading logs

```
docker compose logs -f          # everything
docker compose logs -f ws-server # one service
docker compose logs -f judge-worker # judging problems
```